

UNIVERSITÀ DI BOLOGNA



School of Engineering
Master Degree in Automation Engineering

Optimal Control

Optimal Control of an Actuated Flexible Surface

Professor: **Giuseppe Notarstefano**

Students:
De Blasi Alessandro
Kavishe Kewalramani
Romeo Simone

Academic year 2024/2025

Abstract

This project implements optimal control strategies for an under-actuated flexible surface. We discretize the system dynamics, generate optimal trajectories using Newton-based methods, and design LQR/MPC controllers for trajectory tracking. The implementation includes numerical computation of equilibria, trajectory generation, and visualization through system animation.

Contents

Introduction	5
1 Problem Setup	7
1.1 Dynamics' Discretization	7
2 Trajectory generation	8
2.1 Non-smooth desired trajectory	8
2.2 Implementation	9
2.3 Smooth trajectory generation	12
3 Trajectory Tracking	16
3.1 LQR Tracking	16
3.1.1 Implementation	16
3.1.2 Results	17
3.1.3 Animation	20
3.2 MPC Tracking	21
3.2.1 Implementation	21
3.2.2 Results	23
3.3 Comparison: LQR vs MPC tracking	26
Conclusions	28

Introduction

The project aims to control an underactuated flexible surface. The system is composed of four points in the middle, while the endpoints are fixed. The four points have vertical displacements and only two of them are actuated (p_2, p_4). The model is represented in Figure 1:

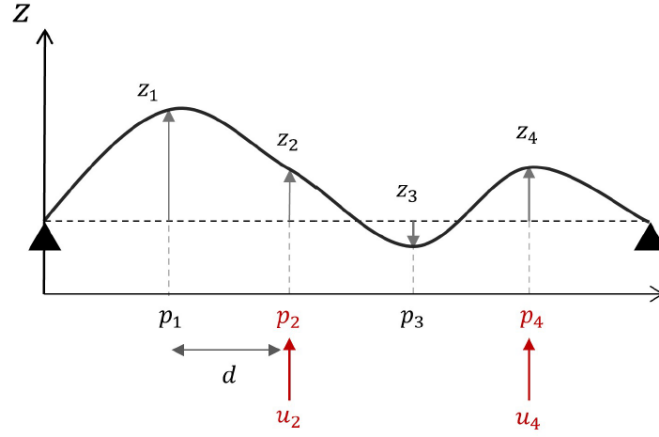


Figure 1: Model of the system

The state space consists of

$$x = [z_1, z_2, z_3, z_4, \dot{z}_1, \dot{z}_2, \dot{z}_3, \dot{z}_4]^\top \quad (1)$$

where each z_i represents the vertical displacement of one of the points of the surface. It is assumed that there are no on-plane forces and stiffness interaction between the points of the surface only produces vertical forces. As we can see, dynamics are represented through second-order differential equations, embedding mass, damping, and mechanical coupling forces:

$$\ddot{z}_i = \frac{1}{m_i} \left[F_i - \alpha \sum_{j \in A} \frac{z_i - z_j}{L_{ij} (L_{ij}^2 - (z_i - z_j)^2)} - c \dot{z}_i \right] \quad (2)$$

Where $F_i = u_i$ only for the actuated points, and $F_i = 0$ for all the others. A is the set of all the points indexes, L_{ij} is the distance between points i and j and α is a mechanical coupling coefficient. All the points are equally spaced, and the distance between direct neighbors is d . All the parameters of the model are available in Table (1.1):

Parameters: Set 3	
m	0.1
m_{act}	0.4
d	0.30
α	$128 \cdot 0.15$
c	0.1

Table 1: Parameters of the model

Chapter 1

Problem Setup

1.1 Dynamics' Discretization

The system's dynamics have been discretized through the Forward Euler method.

After some trials, the discretization step-size has been set to:

$$dt = 10^{-4}$$

With higher values, such as $dt = 10^{-3}$, the system would explode after some iterations, while with lower values, as $dt = 10^{-5}$, it just takes a large amount of time to compute (as it would be more precise, it needs more computation time). The discrete-time state-space are:

$$z_1 = z_1 + dt \cdot \dot{z}_1$$

$$z_2 = z_2 + dt \cdot \dot{z}_2$$

$$z_3 = z_3 + dt \cdot \dot{z}_3$$

$$z_4 = z_4 + dt \cdot \dot{z}_4$$

$$\begin{aligned}\dot{z}_1 &= \dot{z}_1 + dt \cdot \left[\frac{1}{m} \left(\alpha \sum_{j \in A} \frac{z_1 - z_j}{L_{1j}(L_{1j}^2 - (z_1 - z_j)^2)} - c\dot{z}_1 \right) \right] \\ \dot{z}_2 &= \dot{z}_2 + dt \cdot \left[\frac{1}{m_{act}} \left(u_2 - \alpha \sum_{j \in A} \frac{z_2 - z_j}{L_{2j}(L_{2j}^2 - (z_2 - z_j)^2)} - c\dot{z}_2 \right) \right] \\ \dot{z}_3 &= \dot{z}_3 + dt \cdot \left[\frac{1}{m} \left(\alpha \sum_{j \in A} \frac{z_3 - z_j}{L_{3j}(L_{3j}^2 - (z_3 - z_j)^2)} - c\dot{z}_3 \right) \right] \\ \dot{z}_4 &= \dot{z}_4 + dt \cdot \left[\frac{1}{m_{act}} \left(u_4 - \alpha \sum_{j \in A} \frac{z_4 - z_j}{L_{4j}(L_{4j}^2 - (z_4 - z_j)^2)} - c\dot{z}_4 \right) \right]\end{aligned}$$

Chapter 2

Trajectory generation

2.1 Non-smooth desired trajectory

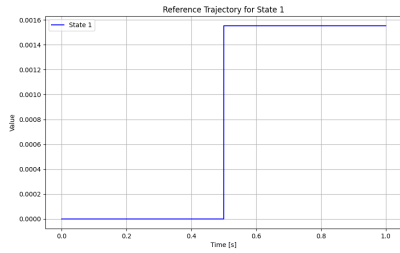
The starting point was to compute two equilibrium points of the system and to design a reference trajectory between them. Those points were determined by solving the equilibrium equations of the system:

$$F_i - \alpha \sum_{j \in A} \frac{z_i - z_j}{L_{ij}(L_{ij}^2 - (z_i - z_j)^2)} = 0 \quad (2.1)$$

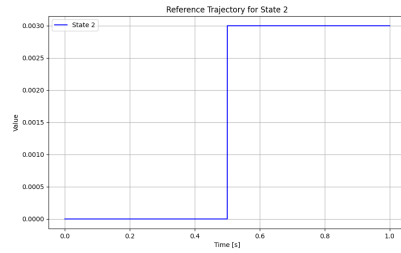
where:

$$F_i = \begin{cases} 0 & \text{for non-actuated points } (i = 1, 3) \\ u_i & \text{for actuated points } (i = 2, 4) \end{cases}$$

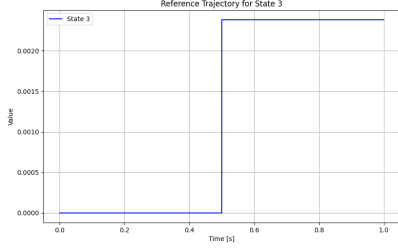
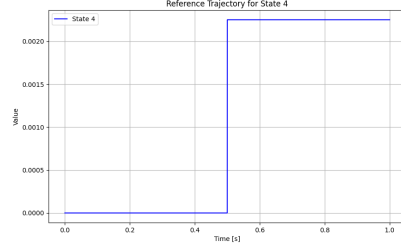
To ensure a structured transition, the trajectory is designed to include two long constant sections at the equilibrium points with a symmetric transition in between.



(a) z_1 reference



(b) z_2 reference

(a) z_3 reference(b) z_4 reference

The equilibrium are computed assigning a desired position to the actuated points. Then, it is necessary to solve a problem in 4 unknowns using the equilibrium equations (exact problem) to find the equilibrium for the non-actuated points and the inputs. This ensure that the equilibrium is consistent and achievable.

2.2 Implementation

Newton's method was employed to accurately determine the equilibrium points and find a feasible and optimal trajectory. Following the equilibrium identification, the transition was optimized by adjusting the control inputs through a Newton optimization approach, aimed at minimizing deviations from the desired path.

- **Gradient Descent Cost Matrices Choice:** After some trials, we chose the following values for the cost matrices:

$$\mathbf{Q}_t = \begin{pmatrix} 100 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 100 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1000 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.1 \end{pmatrix}$$

$$\mathbf{R}_t = \begin{pmatrix} 0.001 & 0 \\ 0 & 0.001 \end{pmatrix}$$

It is possible to notice that the actuated points' positions are weighted more with respect to the non-actuated ones.

- **Cost Evaluation:** The stage cost and terminal cost are computed using quadratic functions. The overall cost is accumulated in time.

- **Backward Riccati Recursion:** Starting from the terminal cost, the Riccati equations are solved backwards in time. In each backward step, the gradients and Hessians of the cost and the linearizations of the dynamics are computed. This produces the feedback gains K_t and the affine terms σ_t .
- **Computation of Descent Direction:** A descent direction d^k is obtained by propagating the linearized dynamics forward in time.
- **Armijo Stepsize Selection:** At each iteration of the method, the stepsize is computed through Armijo rule. This guarantees sufficient decrease of the cost.

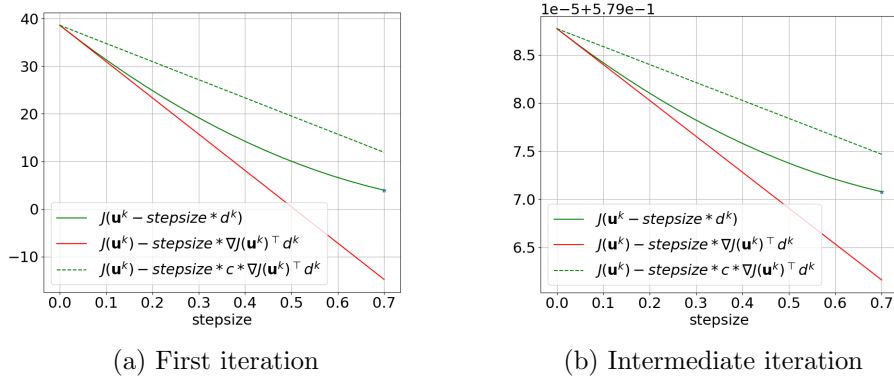


Figure 2.3: Armijo graph

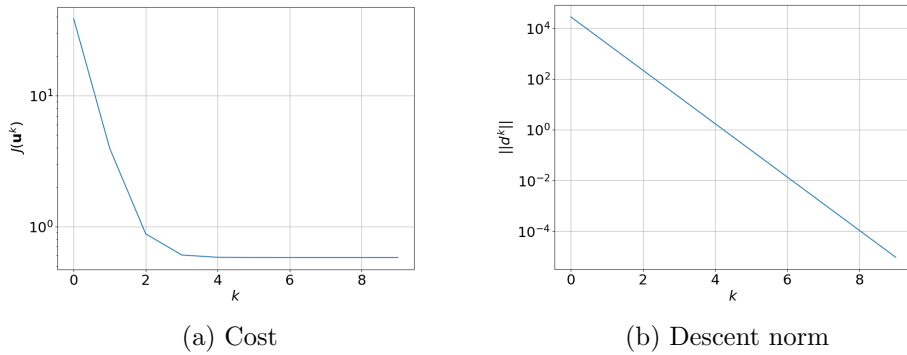


Figure 2.4: Cost and Descent norm

- **Forward Propagation:** The input is updated and the system dynamics are integrated forward in time to obtain the new trajectory. The initial guess (x_0, u_o) for the Newton's method is zero.

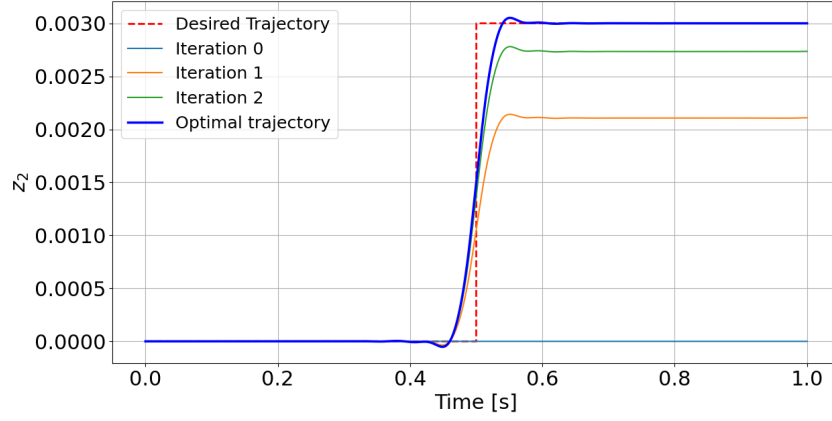
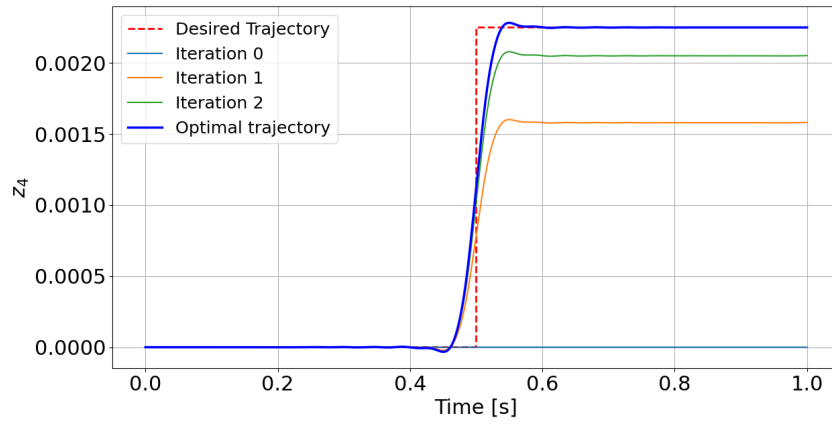
(a) Newton's Method for z_2 (b) Newton's Method for z_4

Figure 2.5: Newton's method for actuated points

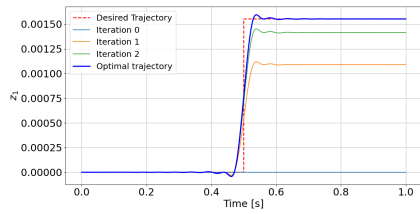
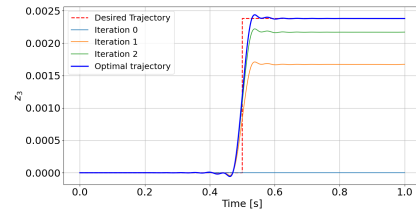
(a) Newton's Method for z_1 (b) Newton's Method for z_3

Figure 2.6: Newton's method for non-actuated points

Then, the algorithm terminates when the norm of the descent direction

falls below the threshold 10^{-6} .

It is possible to notice that as the trajectories become optimal, the descent and the cost decrease quickly with each iteration.

2.3 Smooth trajectory generation

The objective of the second task was to generate a smooth state-input desired trajectory and perform optimal trajectory generation based on the reference curve. To achieve this goal, the same approach as in the non-smooth trajectory is used. Some equilibria has been computed and then interpolated through cubic splines for the four points. This trajectory served as reference for the optimization process performed with Newton's method.

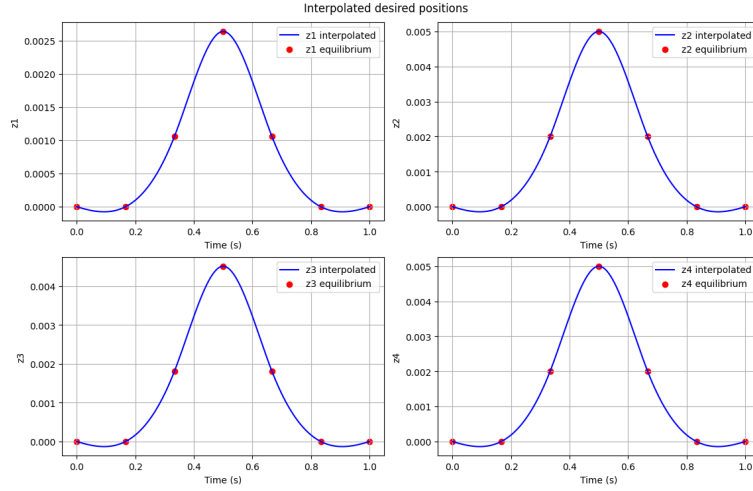


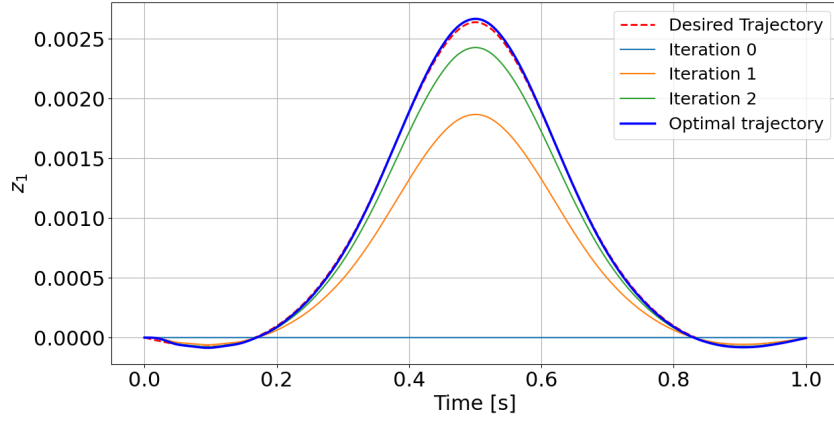
Figure 2.7: Reference smooth positions trajectory

In this task, also some changes were made to the gradient descent cost matrices, that became the following:

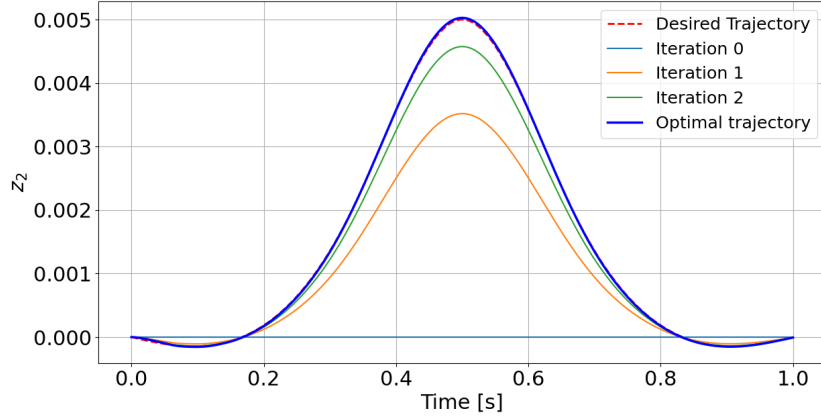
$$\mathbf{Q}_t = \begin{pmatrix} 1000 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1000 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1000 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.1 \end{pmatrix}$$

$$\mathbf{R}_t = \begin{pmatrix} 0.01 & 0 \\ 0 & 0.01 \end{pmatrix}$$

The initial guess $(\mathbf{x}_0, \mathbf{u}_0)$ for the Newton's method is zero. This method allowed iterative refinement of the trajectory, minimizing deviations from the desired reference curve while maintaining smoothness. Iteration after iteration, an optimal trajectory for each point is computed, giving more weight to the positions.

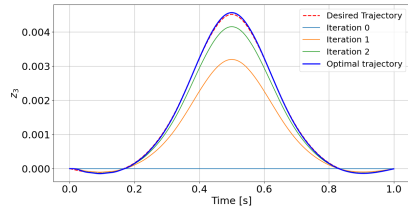
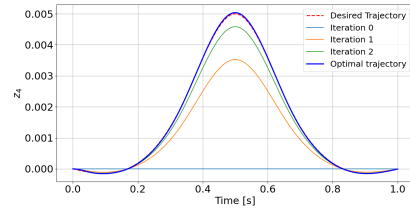


(a) Newton's Method for z_1

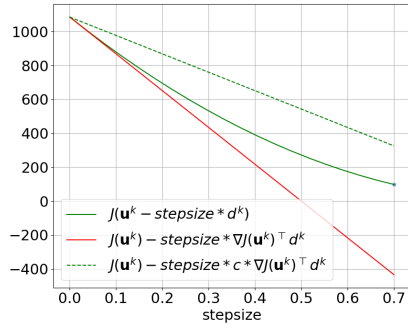


(b) Newton's Method for z_2

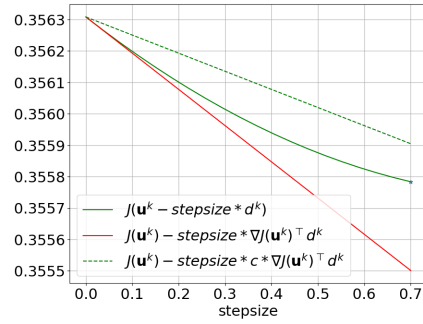
Figure 2.8: Newton's Method Iterations (z_1 and z_2)

(a) Newton's Method for z_3 (b) Newton's Method for z_4 Figure 2.9: Newton's Method Iterations (z_3 and z_4)

For each iteration of the method, Armijo rule is used to select the step-size that permits the greatest decrease of the cost.



(a) First iteration



(b) Intermediate iteration

Figure 2.10: Armijo rule

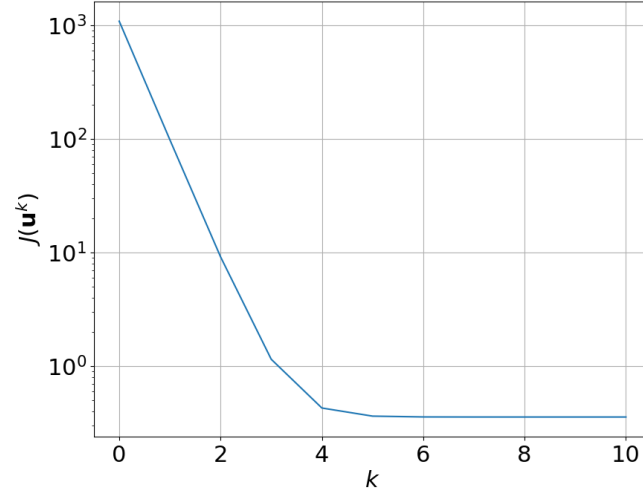


Figure 2.11: Cost

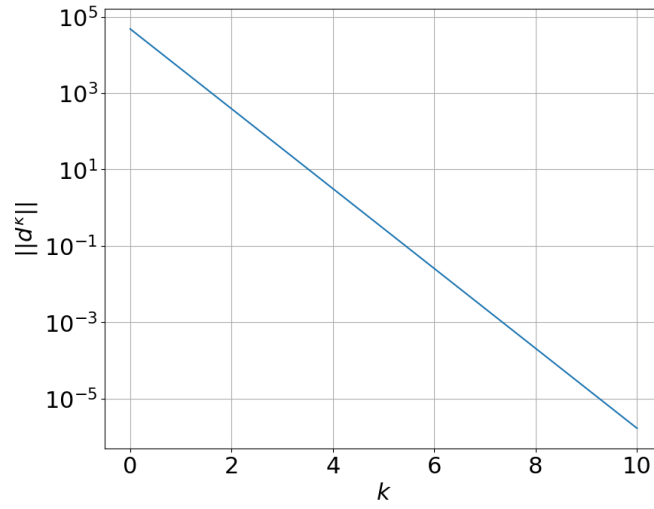


Figure 2.12: Descent norm

The LQR-based feedback matrix was computed by solving the associated Riccati equation for the linearized system, ensuring stability and optimal control effort. The resulting trajectory provided a continuous and well-structured transition between the equilibrium points.

Chapter 3

Trajectory Tracking

3.1 LQR Tracking

The objective was to design a LQR-based controller to track the optimal trajectory $(\mathbf{x}^{\text{traj}}, \mathbf{u}^{\text{traj}})$, that was computed in the previous step. The goal was to solve the finite-horizon LQ optimal control problem to find the feedback gain that minimizes the deviation from the reference, ensuring also robustness to perturbed initial conditions. The problem is:

$$\begin{aligned}
 & \min_{\substack{\Delta x_1, \dots, \Delta x_T \\ \Delta u_0, \dots, \Delta u_{T-1}}} \sum_{t=0}^{T-1} \Delta x_t^\top Q_t^{\text{reg}} \Delta x_t + \Delta u_t^\top R_t^{\text{reg}} \Delta u_t + \Delta x_T^\top Q_T^{\text{reg}} \Delta x_T \\
 & \text{subj. to } \Delta x_{t+1} = A_t^{\text{traj}} \Delta x_t + B_t^{\text{traj}} \Delta u_t, \quad t = 0, \dots, T-1 \\
 & x_0 = x_{\text{init}}
 \end{aligned} \tag{3.1}$$

3.1.1 Implementation

The LQR tracking implementation follows these steps:

- **Reference Trajectory Loading:** The optimal trajectory $(\mathbf{x}^{\text{traj}}, \mathbf{u}^{\text{traj}})$, generated via Newton-based optimization in previous step is loaded from saved data files to ensure a faster execution of the code.
- **Linearization:** At each time step, the Jacobians of the system dynamics $(\nabla_x f_t(x_t^{\text{traj}}, u_t^{\text{traj}})^T, \nabla_u f_t(x_t^{\text{traj}}, u_t^{\text{traj}})^T)$ are computed numerically using symbolic differentiation. These Jacobians defined time-varying matrices A_t^{traj} and B_t^{traj} .
- **Riccati Backward Iteration:** The discrete-time Riccati equation is solved backward in time to compute the time-varying feedback gains K_t^{reg} .

- **Feedback Control Law:** The control input is computed as:

$$u_t = u_t^{\text{traj}} + K_t^{\text{reg}}(x_t - x_t^{\text{traj}}), \quad (3.2)$$

where K_t^{reg} , found in the previous step, corrects deviations from the reference trajectory.

- **LQR Cost Matrices Choice:** After some trials, we chose the following values for the cost matrices:

$$\mathbf{Q}_t = \begin{pmatrix} 1000 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1000 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1000 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.1 \end{pmatrix}$$

$$\mathbf{R}_t = \begin{pmatrix} 0.001 & 0 \\ 0 & 0.001 \end{pmatrix}$$

The goal was to introduce a major penalization on positions and a smooth control for the inputs.

- **Perturbed Initialization:** The system is initialized with a perturbed state to test its robustness. The perturbation is of the same order of magnitude of the trajectory one:

$$x_0 = [-0.002 \quad -0.002 \quad -0.002 \quad -0.002 \quad 0 \quad 0 \quad 0 \quad 0]$$

- **Tracking Error Analysis:** Position errors ($x_t - x_t^{\text{traj}}$) and input deviations are plotted to quantify the tracking performance.

3.1.2 Results

The LQR controller successfully tracked the reference trajectory despite initial perturbations.

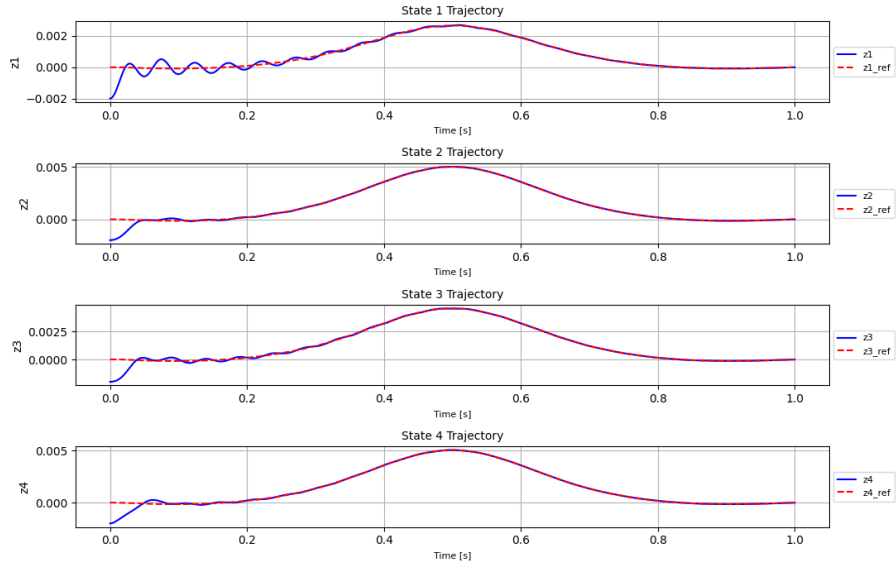


Figure 3.1: LQR tracking positions

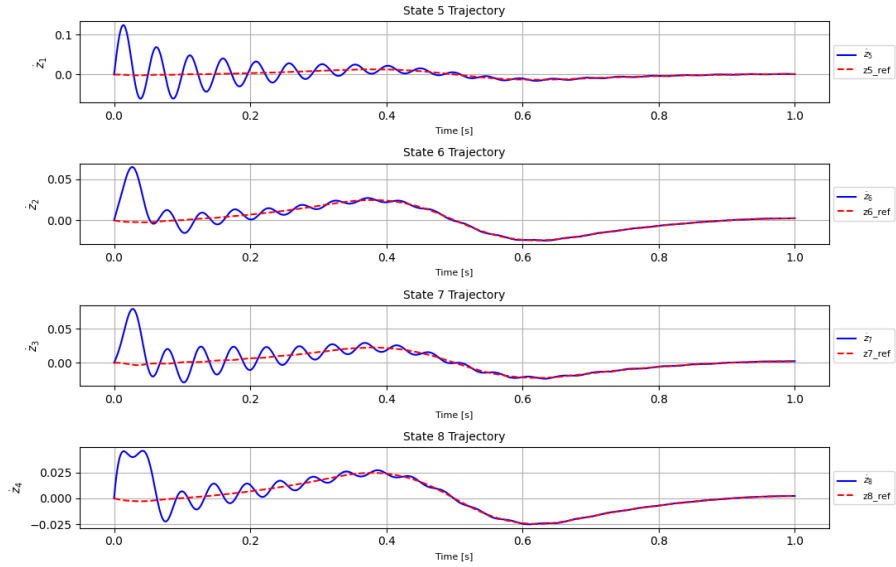


Figure 3.2: LQR tracking velocities

Taking into account all points of the surface, the key outcomes included the following:

- **Robust Position Tracking:** In particular:
 - Maximum Position Error (on the second half of the simulation): 3.87×10^{-5} m

- Mean Position Error (on the whole simulation):
 8.02×10^{-5} m
- Mean Position Error (on the first half of the simulation):
 1.56×10^{-4} m
- **Robust Velocity Tracking:** In particular:
 - Maximum velocity error (on the second half of the simulation):
 5.38×10^{-3} m/s
 - Mean velocity error (on the whole simulation):
 6.34×10^{-3} m/s
 - Mean velocity error (on the first half of the simulation):
 1.21×10^{-2} m/s
- **Convergence Time** (only positions, with a threshold of 10% of the maximum initial error): 0.22 s

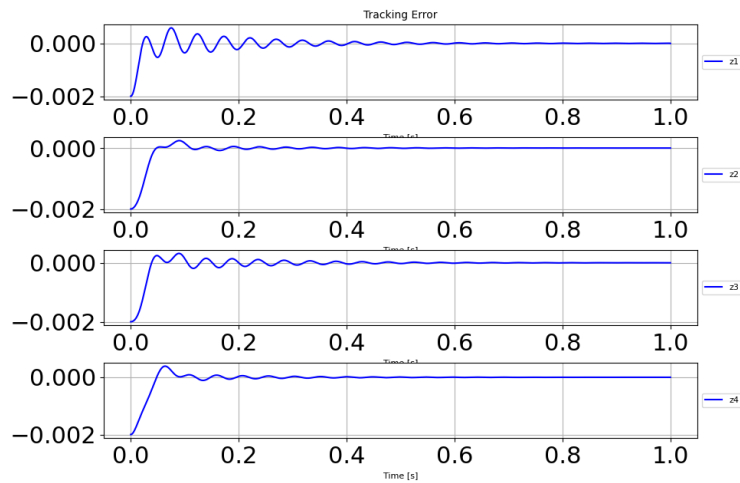


Figure 3.3: LQR tracking errors

- The convergence of tracking errors to near-zero values is reached even for different perturbed initial conditions.

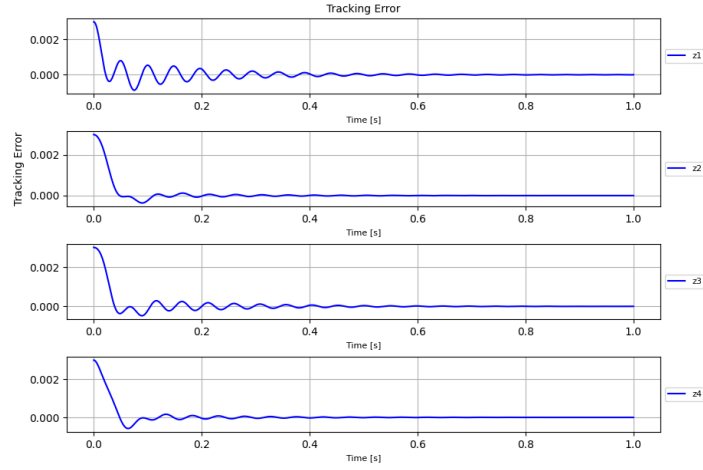


Figure 3.4: LQR tracking errors ($x_0 = [0.003, 0.003, 0.003, 0.003, 0, 0, 0, 0]$)

- Smooth control inputs that follow the reference u_t^{traj} with initial oscillations to correct the position.

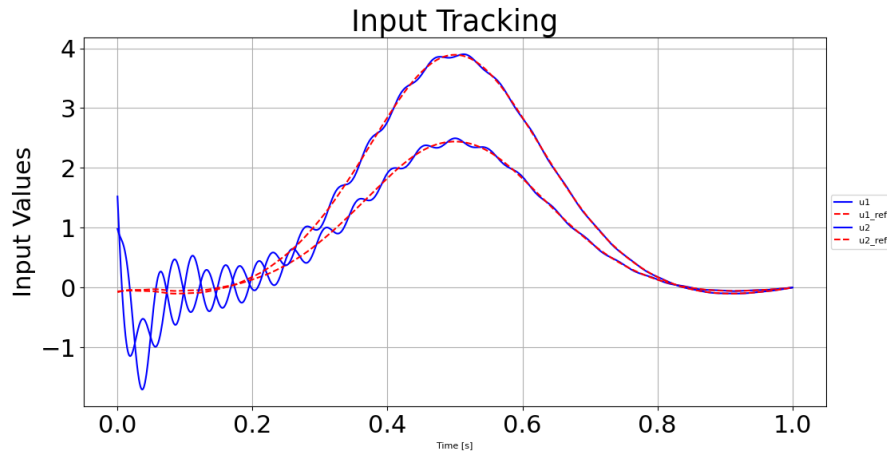


Figure 3.5: LQR tracking inputs

3.1.3 Animation

An animation is used to directly visualize the behavior of the surface while tracking the desired trajectory. It can be seen that the surface begins to track perfectly the trajectory pretty fast.

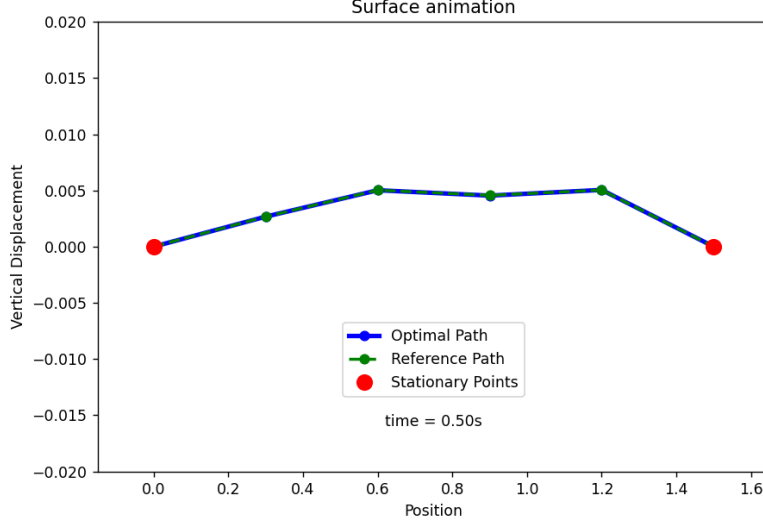


Figure 3.6: Animation frame

3.2 MPC Tracking

Differently from LQR Tracking, the MPC Tracking approach is in real time and emphasizes the handling of constraints. A quadratic programming problem (QP) is solved at each iteration with a determined time horizon to compute the input for the system.

3.2.1 Implementation

The MPC tracking implementation followed these steps:

- **Prediction-Horizon Selection:** A prediction horizon T_{pred} was chosen based on system's dynamics to balance performance and computational cost. Longer horizons improve performance but increase computational complexity.
- **Reference Trajectory Management:** The precomputed desired optimal trajectory was extended or truncated to match the simulation horizon T_{sim} steps using linear interpolation.
- **Linearization Step:** At each time step, the system was linearized about the current state x_t using Jacobians computed in the dynamics code. From this step, the matrices A_t, B_t were obtained.

- **MPC Cost Matrices Choice:** After some trials, we chose the following values for the cost matrices:

$$\mathbf{Q}_t = \begin{pmatrix} 1000 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1000 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1000 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 500 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 500 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 500 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 500 \end{pmatrix}$$

$$\mathbf{R}_t = \begin{pmatrix} 0.01 & 0 \\ 0 & 0.01 \end{pmatrix}$$

Again, as in the LQR tracking, the goal was to introduce a major penalization on positions and a smooth control for the inputs.

- **Constraints:** Input constraints can be set easily, modifying u_{max}
- **Perturbed Initialization:** The system is initialized with a perturbed state to test robustness. The perturbation is of the same magnitude order of the surface reference trajectory:

$$x_0 = [-0.002 \quad -0.002 \quad -0.002 \quad -0.002 \quad 0 \quad 0 \quad 0 \quad 0]$$

Notice that the perturbation is the same as the LQR Tracking one, to allow for comparisons.

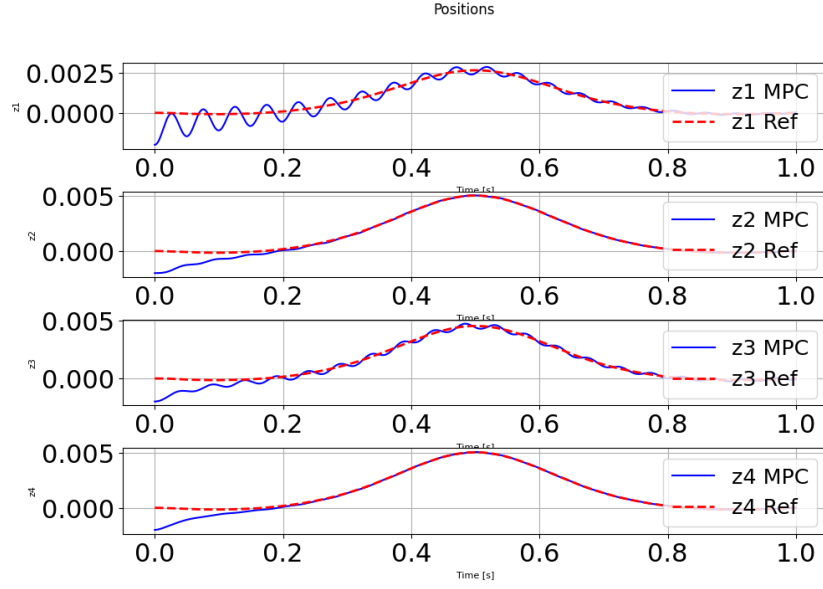
- **Constrained QP Formulation:** For each time t and given T_{pred} , the prediction horizon, the following optimization problem is solved:

$$\begin{aligned} & \min_{x_t, \dots, x_{t+T}, u_t, \dots, u_{t+T-1}} \sum_{\tau=t}^{t+T-1} \ell_{\tau}(x_{\tau}, u_{\tau}) + \ell_{t+T}(x_{t+T}) \\ & \text{subj. to} \\ & x_{\tau+1} = f(x_{\tau}, u_{\tau}) \quad \forall \tau = t, \dots, t+T-1 \\ & x_{\tau} \in \mathcal{X}, u_{\tau} \in \mathcal{U} \quad \forall \tau = t, \dots, t+T \\ & x_t = x_t^{\text{meas}} \end{aligned} \tag{3.3}$$

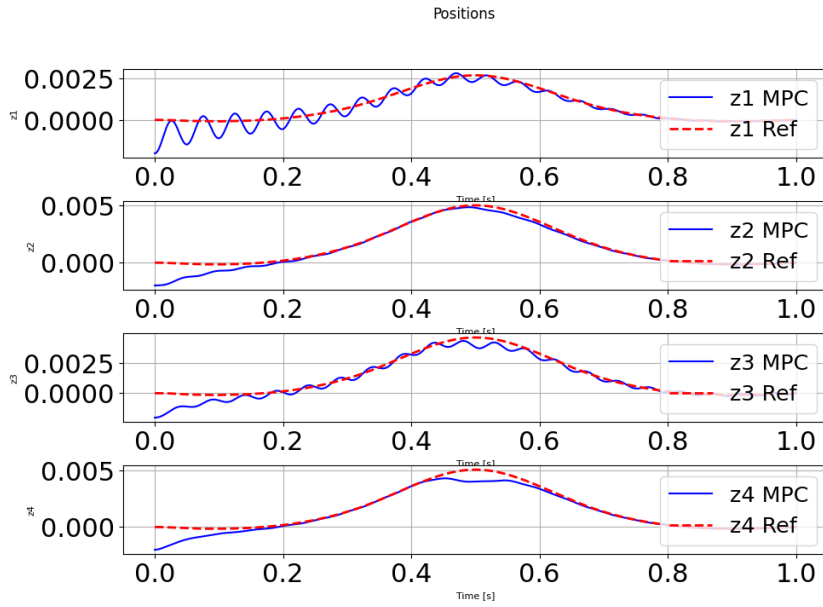
To increase computation speed, a choice was taken to not solve the QP at each iteration, but only every time the iteration number is divisible by qp_solve_freq . This feature can be removed by setting $qp_solve_freq = 1$.

3.2.2 Results

The reference trajectory is successfully tracked. Also a simulation with a tighter constraint on the input has been performed ($u_{\max} = 3$),



(a) $u_{\max} = 5$



(b) $u_{\max} = 3$

Figure 3.7: MPC position tracking with different constraints on u

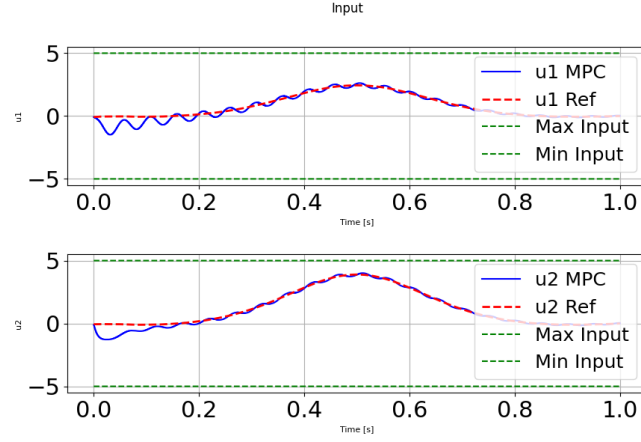
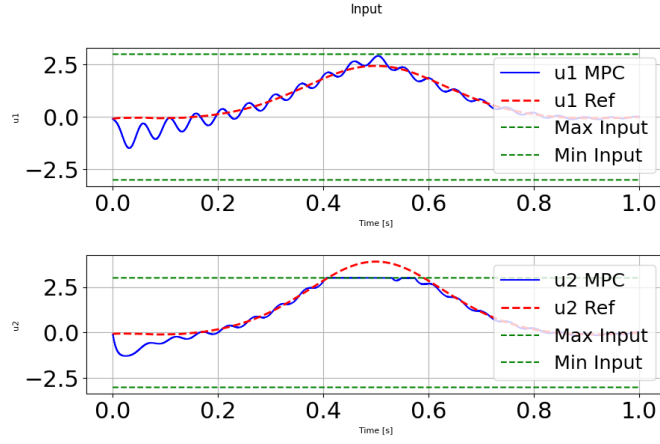
(a) $u_{\max} = 5$ (b) $u_{\max} = 3$

Figure 3.8: MPC input tracking with different constraints

Considering all the points of the surface, the MPC implementation achieved:

- **Robust Position Tracking:** In particular:
 - Maximum Position Error (on the second half of the simulation): 2.41×10^{-4} m
 - Mean Position Error (on the whole simulation): 1.97×10^{-4} m
 - Mean Position Error (on the first half of the simulation): 3.46×10^{-4} m
- **Robust Velocity Tracking:** In particular:
 - Maximum velocity error (on the second half of the simulation): 3.22×10^{-2} m/s

- Mean velocity error (on the whole simulation):
 1.23×10^{-2} m/s
- Mean velocity error (on the first half of the simulation):
 1.83×10^{-2} m/s
- **Convergence Time** (only positions, with a threshold of 10% of the maximum initial error: 0.61 s

It can be seen that for the non actuated points the evolution of the error is a little wavy, but it converges to zero anyway:

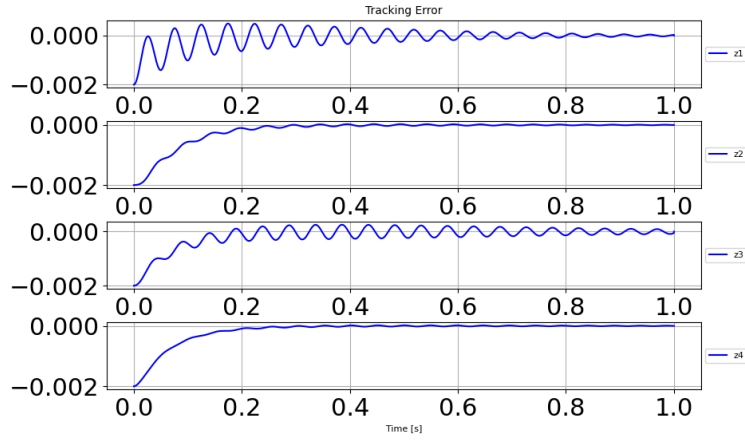


Figure 3.9: MPC Tracking error ($u_{max} = 5$)

Even for different perturbed initial condition, the error converges to zero.

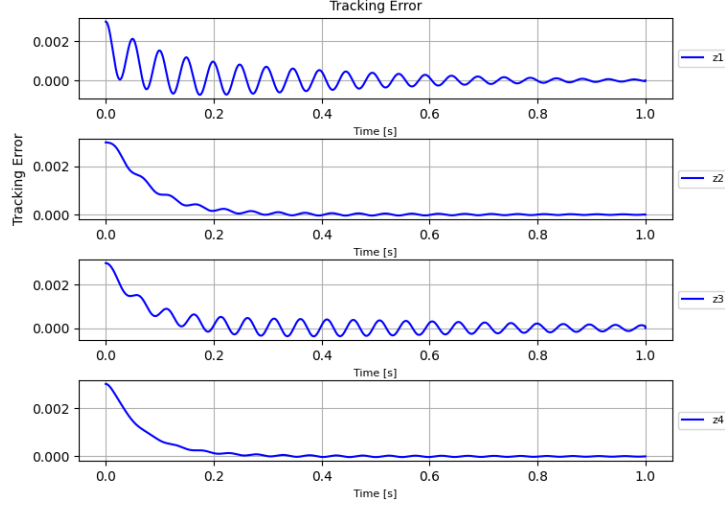


Figure 3.10: MPC Tracking error ($x_0 = [0.003, 0.003, 0.003, 0.003, 0, 0, 0, 0]$)

3.3 Comparison: LQR vs MPC tracking

The following observations have been made on the same perturbed initial condition for both algorithms:

$$x_0 = [-0.002 \quad -0.002 \quad -0.002 \quad -0.002 \quad 0 \quad 0 \quad 0 \quad 0]$$

It can be observed that LQR provides a faster and smoother trajectory convergence compared to MPC ($u_{max} = 5$) when the initial state deviates from the reference. MPC, due to its iterative nature and horizon limitations, may introduce oscillations, while LQR ensures a steady and direct feedback response. It is possible to observe what follows:

- **Convergence Velocity:** LQR tracking is faster than MPC tracking in obtaining the convergence. Assuming convergence when the tracking error stays below a threshold (10% of the initial error) for at least 0.2s on a trajectory lasting 1 second, it can be seen that:
 - LQR convergence time is : 0.22s
 - MPC convergence time is : 0.61s
- **Position Tracking:** Comparison of position tracking errors considering all four points:

	$\max(err)_{2ndhalf}$	$\text{mean}(err)$	$\text{mean}(err)_{1sthalf}$
LQR	3.87×10^{-5}	8.02×10^{-5}	1.56×10^{-4}
MPC	2.41×10^{-4}	1.97×10^{-4}	3.46×10^{-4}

Table 3.1: Position Tracking Errors

- **Velocity Tracking:** Comparison of velocity tracking errors considering all four points:

	$\max(err)_{2ndhalf}$	$\text{mean}(err)$	$\text{mean}(err)_{1sthalf}$
LQR	5.38×10^{-3}	6.34×10^{-3}	1.21×10^{-2}
MPC	3.22×10^{-2}	1.23×10^{-2}	1.83×10^{-2}

Table 3.2: Velocity Tracking Errors

Conclusions

This project focused on trajectory tracking for a flexible surface using two advanced control strategies: Linear Quadratic Regulator (LQR) and Model Predictive Control (MPC). The primary objective was to design controllers that could accurately track a precomputed optimal trajectory while ensuring robustness against perturbations. The project work followed a structured approach, starting with trajectory optimization using Newton's method, followed by LQR and MPC implementations, and concluding with a comparative analysis of both methods.

Initially, an optimal trajectory was generated using a Newton-based optimization approach. This trajectory, consisting of state and control input sequences, was stored to facilitate efficient controller implementation. The LQR-based tracking method was then developed to minimize deviations from this reference. The system was linearized at each time step using Jacobians, and the discrete-time equation was solved backward to obtain optimal feedback. The results showed that the LQR controller provided a smooth and stable trajectory tracking with rapid error convergence, even when initialized with perturbations.

The MPC-based tracking approach was subsequently implemented, leveraging real-time optimization to handle system constraints. A quadratic programming problem was solved iteratively over a predefined prediction horizon, ensuring that the control inputs remained within physical limits. The MPC controller successfully tracked the reference trajectory while maintaining actuator constraints, although it exhibited slightly higher tracking errors compared to LQR due to its iterative nature. A comparison of LQR and MPC showed that LQR offered faster convergence and smoother control actions, making it computationally efficient for applications where real-time constraint handling is not a priority. Conversely, MPC proved advantageous in scenarios requiring explicit constraint enforcement, Eg. at the cost of increased computational complexity. The results showed that LQR achieved superior position tracking, while MPC provided better adaptability to constraints. In conclusion, this project successfully implemented and analyzed LQR and MPC controllers for trajectory tracking on a flexible surface.

Bibliography

- [1] Notarstefano, G. *Optimal Control Lecture Notes*. UniBO, 2024.